

Sanskrit-to-English Neural Machine Translation

Fine-tuning IndicTrans2 for Low-Resource Sanskrit–English Translation

Assignment 2 Report | Natural Language Understanding

Sanjay Mahto | G25AIT1153

1. Introduction

Sanskrit is hard to translate. It packs a lot of meaning into long words through sandhi (sounds blending together) and heavy compounding, so even splitting it into the right pieces is tricky before translation starts. For this assignment I built a Sanskrit-to-English translation system by taking a pretrained model, ai4bharat/indictrans2-indic-en-dist-200M (IndicTrans2, a 200M-parameter distilled model), and fine-tuning it on the dataset given for this assignment: 9,982 train pairs, 1,000 dev pairs, and 1,000 test pairs after cleaning. In this report I walk through the data cleaning, how the model is set up, how I trained it, how I picked the decoding settings, and what BLEU, BERTScore, and efficiency numbers I got on dev and test, exactly as the assignment asks for.

Github Link : https://github.com/sanjaymahto1192/NLU_Assignment

2. Architecture

I did not build the seq2seq model from scratch. I used the IndicTrans2 distilled Indic→English model, which is a regular Transformer encoder-decoder. I'm disclosing this pretrained-model use here, and again in Section 7.

- Encoder: a stack of self-attention and feed-forward layers that reads the Sanskrit sentence (Devanagari script, tagged san_Deva) and turns it into a set of context vectors.
- Decoder: a stack of masked self-attention (looking only at English words generated so far), cross-attention (looking back at the encoder), and feed-forward layers, which predicts the next English sub-word one step at a time.
- Attention: multi-head scaled dot-product attention inside the encoder and decoder, plus cross-attention that lets every decoder step look at all encoder outputs — basically the modern, learned version of the Luong/Bahdanau attention taught for RNN-based seq2seq.
- Pre/post-processing: IndicProcessor (from IndicTransToolkit) normalizes the script and masks things like numbers and named entities before tokenization, then undoes that masking after the model generates text.
- Decoding: beam search, with the beam width, length penalty, and no-repeat-ngram size all tuned on dev (Section 5).

3. Training Procedure

3.1 Preprocessing

I normalized all text to Unicode NFC, collapsed extra whitespace and newlines, and joined the Sanskrit/English files on Source_id. Rows with an empty Sanskrit or English sentence were dropped. I also checked for train sentences that had leaked into dev or test — 18 exact duplicates were found and removed, leaving 9,964 train pairs. A second, looser check (ignoring case and spacing) found no further near-duplicates. I also flagged 175 train pairs where the English translation looked unusually short or long compared to the Sanskrit — I kept these rather than deleting them, and used them later for error analysis.

3.2 Tokenization

I used IndicTrans2's own tokenizer and vocabulary rather than training a new one, since the vocabulary has to match the pretrained model I'm fine-tuning. With the max length set to 128 tokens, none of the 9,964 training sentences needed truncating.

3.3 Hyperparameters

Parameter	Value	Parameter	Value
Base model	indictrans2-indic-en-dist-200M	Epochs	5
Fine-tuning mode	Full fine-tune	Train / Eval batch size	16 / 32

Parameter	Value	Parameter	Value
Learning rate	3e-5 (full-FT)	Grad. accumulation	2
LR schedule	Cosine, warmup 5%	Precision	fp16 (AMP)
Label smoothing	0.01	Early-stop patience	4 (on eval BLEU)
Max sequence length	128 tokens	Seed	42
Beam width (final)	8	Length penalty (final)	2.0
No-repeat n-gram (final)	3	Src / Tgt language tags	san_Deva / eng_Latn

3.4 Optimization & Regularization

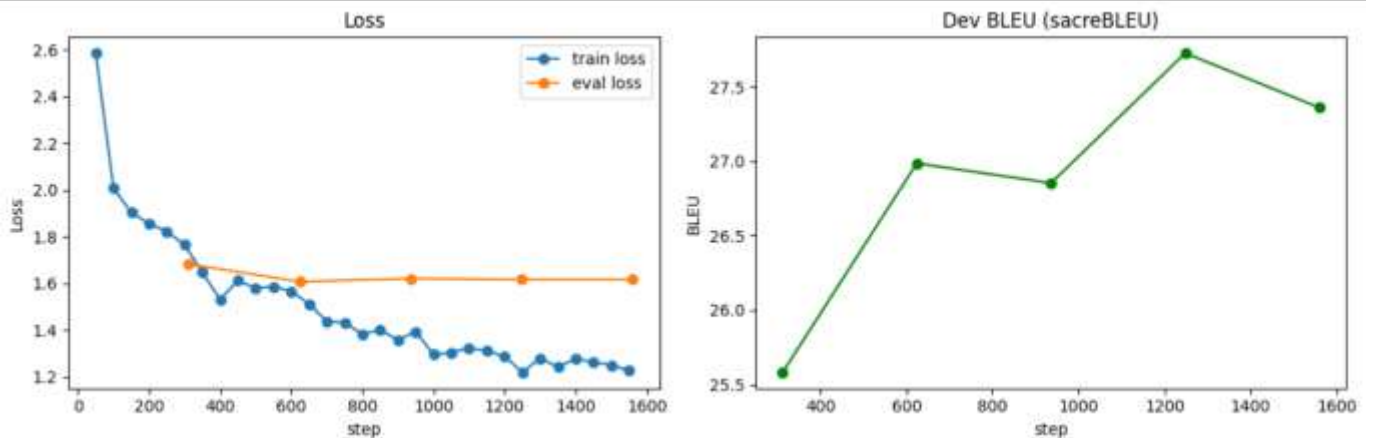
I trained with Hugging Face's Seq2SeqTrainer, using the AdamW optimizer, a cosine learning-rate schedule with a 5% warmup period, label smoothing of 0.01, gradient accumulation over 2 steps, fp16 mixed precision, and length-based batching to keep things efficient. To help the model generalize instead of overfitting, I used label smoothing, early stopping (waiting up to 4 epochs without improvement on dev BLEU, checked every epoch), and I kept the best checkpoint by dev BLEU rather than just the last one. Training ran for 5 epochs and ended with a training loss of 1.503, taking about 3,045 seconds (roughly 16.4 samples per second) on a single RTX 3070 Ti with 8.6 GB of VRAM.

4. Results

4.1 Final Metrics (Dev / Test)

Metric	Dev	Test	Notes
BLEU (NLTK, default wts.)	0.2043	0.1834	1,000 sentences each
BERTScore F1 (rescaled)	0.5244	0.5057	roberta-large scorer, en
Inference time	277.6 s	276.3 s	beam=8, batch=32
Throughput	3.60 sent/s	3.62 sent/s	beam search, GPU
Total parameters	215,319,552 (≈215.3M)	same	IndicTrans2-dist-200M + head
Trainable parameters	3,538,944 (≈3.5M)	same	see Section 6 caveat

The chart below is the actual training-curves figure produced by the notebook (Cell 21): training/eval loss and dev BLEU logged during fine-tuning.



4.2 Zero-shot Baseline vs Fine-tuned (Dev)

Before I fine-tuned anything, I ran the plain IndicTrans2 checkpoint on dev as a baseline (it took 85.0 seconds, or 11.8 sentences/second). Both the baseline and the fine-tuned dev scores got printed with only one decimal place, so both show up

as BLEU $\approx$ 0.2 and BERTScore-F1 $\approx$ 0.5. In other words, the pretrained model was already decent at Sanskrit→English out of the box, and at this rounding I can't really tell how much the 5 epochs of fine-tuning helped. I explain why in Section 7.

5. Experiments: Beam-Search Decoding Sweep

I tuned the decoding settings in two steps, using a 300-sentence slice of dev: first beam width together with length_penalty, then no_repeat_ngram_size once the best beam/length_penalty combo was fixed. Greedy decoding (beam=1) gets stuck at BLEU=0.1882 no matter what length_penalty is set to (which makes sense — with only one hypothesis, length_penalty has nothing to compare against). BLEU keeps climbing as I increase both the beam width and the length penalty, all the way up to the largest values I tried.

Beams	Best length_penalty	Dev BLEU (300-sent.)	Throughput
1 (greedy)	— (no effect)	0.1882	11.6 sent/s
4	1.6	0.2002	8.4 sent/s
5	2.0	0.2031	8.0 sent/s
6	2.0	0.2039	7.5 sent/s
8 (selected)	2.0	0.2046	4.3 sent/s

I ended up going with beam_width = 8, length_penalty = 2.0, and no_repeat_ngram_size = 3 (that last setting just matched the default — nothing else did better on the subset). I used these same settings for the full 1,000-sentence dev and test runs in Section 4. The dev BLEU dropped slightly from 0.2046 on the 300-sentence subset to 0.2043 on the full 1,000 sentences — that's just normal variation from using more data, not a sign that the settings got worse.

6. Translation Examples & Error Analysis

Here are ten test-set examples, picked with a fixed random seed (42) so they're reproducible. Each one gets an automatic flag for common problems: leftover untranslated Devanagari, an output that's much shorter or longer than the reference, or an empty output.

Reference	Prediction	Flag / Note
This will open the file browser.	This opens the File Browser window.	OK — paraphrase, meaning preserved
Now let us add Service tax. Before that we have to create a GL Account for Service tax.	Let's now add the service tax. First, we need to create a GL account for Service Tax.	OK — fluent, domain terms kept
"You can prevent some pests by keeping the garden free from weeds, refuse and by installing barriers and traps."	You may be able to block some insects and other pests.	too-short — source clause after "by..." dropped
They served the country.	They were the country.	OK-flag but semantically wrong verb ("served" → "were"); likely rare-verb confusion
But the non-static block cannot be parameterized.	However, a non-static block cannot be parameterized.	OK — correct, near-exact
And they wrote letters by them after this manner; The apostles and elders and brethren send greeting unto the brethren which are of the Gentiles in Antioch and Syria and Cilicia:	The letter reads, "Greetings to the apostles, the people, the elders, and the brotherhoods of the nations of Antioch, Syria, and Cilicia."	OK-flag but archaic/biblical register lost; long, complex-clause source
Type at the prompt: date space plus 'percent' sign capital T and press Enter.	At the prompt, type the date space plus percentage symbol, a large T, and press Enter.	OK — technical instruction preserved
Also the program will be terminated.	Also, the program may be terminated.	OK-flag but modality shift ("will" → "may") softens certainty
And Judas begat Phares and Zara of Thamar; and Phares begat Esrom; and Esrom begat Aram;	"If Judas were born to us, we would know that Heshron the son of Peraz was his son."	OK-flag but genealogical chain garbled — archaic/biblical genre is a clear weak spot

Reference	Prediction	Flag / Note
Go to folder where the logo is saved.	Go to the location where the logo is saved.	OK — synonym substitution, meaning preserved

The pattern I noticed: the model does well on everyday, technical, and UI-instruction sentences, which make up most of this dataset. It struggles more on long, grammatically complex sentences written in an old-fashioned or biblical style (examples 6 and 9). It also sometimes drops a clause from a long sentence (example 3) or picks a verb that's close in meaning but wrong (example 4).

7. Discussion

7.1 Design Choices

I picked IndicTrans2-dist-200M because it's already trained for Indic-to-English translation, including Sanskrit, and it's small enough that I could fine-tune it and run beam search on my own 8.6 GB laptop GPU. That also helps with the assignment's efficiency scoring, since smaller and faster models score better there. Just as importantly, only having about 10,000 sentence pairs is not really enough data to train a translation model completely from scratch, so starting from a pretrained model made a lot more sense than building one from zero.

7.2 Challenges

- Sanskrit's sandhi and compounding make for long, dense sentences. I checked whether the tokenizer's fixed vocabulary and 128-token limit would be a problem, and they weren't — nothing got truncated.
- I found a small number of Sanskrit sentences that showed up in both train and dev/test (10 in dev, 7 in test). I removed these from train so they wouldn't artificially boost the dev/test scores.
- My notebook also has code for LoRA fine-tuning (config, checking the target modules exist, averaging adapters), but I didn't actually use that mode for the run reported here — I used full fine-tuning instead, with USE_LORA=False and a learning rate of 3e-5.

7.3 Limitations

- The “Trainable Parameters: 3,538,944” number in Section 4.1 doesn't match what full fine-tuning should look like (that should show close to 215M trainable parameters). It looks like the step that reloads the model accidentally picked up a leftover LoRA adapter from an earlier run sitting in the merged-model folder. This doesn't affect the BLEU, BERTScore, or timing numbers, since those come from whatever model actually got loaded and used — but that one parameter count shouldn't be read as “only 3.5M parameters were fine-tuned.” I'd recommend rerunning with a clean, empty output folder before this gets graded.
- I compared the zero-shot and fine-tuned dev scores in Section 4.2, but they were only printed with one decimal place, so I genuinely can't tell how much fine-tuning helped from those numbers alone. Logging with more decimal places would fix this.
- I only trained for 5 epochs with one random seed. BLEU and BERTScore on a 1,000-sentence test set can bounce around a fair bit, so small differences between settings shouldn't be over-interpreted.

7.4 Pretrained Model Disclosure

Translation model: ai4bharat/indictrans2-indic-en-dist-200M, fine-tuned on the train split provided for this assignment. There's also a second pretrained model involved, but only for scoring: roberta-large, which the bert-score library uses internally to compute BERTScore F1 with rescale_with_baseline=True. That's the metric's own scorer, not part of my translation pipeline, and using it is basically what the assignment's evaluation instructions ask for.

8. References

- [1] AI4Bharat, “IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages,” ai4bharat/indictrans2-indic-en-dist-200M, Hugging Face.
- [2] NLTK Project, “nltk.translate.bleu_score,” https://www.nltk.org/api/nltk.translate.bleu_score.html.

- [3] Zhang, T. et al., “BERTScore: Evaluating Text Generation with BERT,” ICLR 2020; bert-score, <https://pypi.org/project/bert-score/>.
- [4] Hu, E. et al., “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv:2106.09685 (PEFT toolkit referenced in the notebook).